

02 — Differenze tra Kubernetes e OpenShift

Modulo: 1 — Introduzione e Architettura

Versione: OpenShift 4.x su Azure ARO

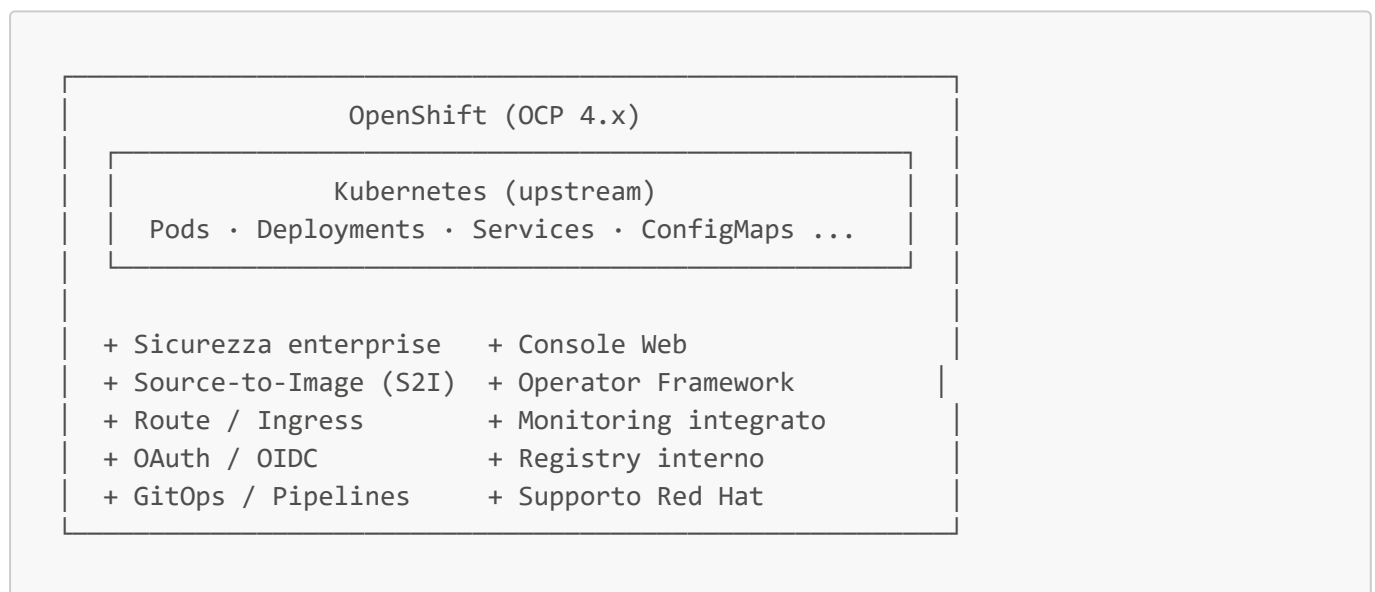
Obiettivi di Apprendimento

Al termine di questa lezione, il partecipante sarà in grado di:

- Comprendere la relazione tra Kubernetes e OpenShift
- Identificare le funzionalità aggiuntive di OCP rispetto a Kubernetes vanilla
- Scegliere consapevolmente OCP per scenari enterprise
- Riconoscere le differenze operative nella gestione quotidiana del cluster

1. La Relazione tra Kubernetes e OpenShift

OpenShift **non è un fork** di Kubernetes: è una **distribuzione enterprise** certificata che include Kubernetes come componente di base, arricchendola con funzionalità aggiuntive.



2. Confronto Diretto: Funzionalità

Funzionalità	Kubernetes Vanilla	OpenShift (OCP 4.x)
Orchestrazione container	Nativa	Nativa (stessa API)
CLI	<code>kubectl</code>	<code>oc</code> (superset di <code>kubectl</code>)
Console Web	Dashboard (non ufficiale)	Console enterprise integrata
Autenticazione	Configurazione manuale	OAuth, OIDC, LDAP, GitHub, AAD

Funzionalità	Kubernetes Vanilla	OpenShift (OCP 4.x)
Autorizzazione RBAC	Nativa	RBAC + SCC aggiuntivi
Container Runtime	containerd (default)	CRI-O (ottimizzato per OCP)
Networking	CNI plugin (scelta libera)	OVN-Kubernetes (default in OCP 4.x)
Ingress	Ingress Controller (configurazione manuale)	Route con TLS out-of-the-box
Registry immagini	Esterno (da configurare)	Registro interno integrato
Build applicazioni	Non incluso	Source-to-Image (S2I) , BuildConfig
CI/CD	Non incluso	OpenShift Pipelines (Tekton)
GitOps	Richiede ArgoCD separato	OpenShift GitOps (ArgoCD integrato)
Operator Framework	OLM opzionale	OLM integrato e pre-configurato
Monitoring	Da configurare (kube-prometheus)	Stack Prometheus/Grafana integrato
Logging	Non incluso	OpenShift Logging (LokiStack)
Aggiornamenti cluster	Manuali	OTA automatizzati con canali
Security Context	PodSecurityAdmission	SCC (SecurityContextConstraints)
Supporto	Community	Red Hat Enterprise Support (SLA)
Certificazione	Non applicabile	EX180, EX280 (Red Hat)

3. Funzionalità Aggiuntive di OCP in Dettaglio

3.1 Route vs Ingress

In Kubernetes la gestione del traffico HTTP/HTTPS in ingresso richiede la configurazione di un **Ingress** object e di un Ingress Controller separato.

OpenShift introduce il concetto di **Route**, più semplice e con TLS gestito nativamente:

```
# Kubernetes – Ingress (richiede Ingress Controller configurato a parte)
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
    - host: myapp.example.com
```

```

http:
  paths:
  - path: /
    pathType: Prefix
    backend:
      service:
        name: my-service
        port:
          number: 8080

```

```

# OpenShift – Route (TLS edge out-of-the-box, nessun controller da configurare)
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: my-route
spec:
  host: myapp.apps.cluster.example.com
  to:
    kind: Service
    name: my-service
  port:
    targetPort: 8080
  tls:
    termination: edge
    insecureEdgeTerminationPolicy: Redirect

```

```

# Creare una Route con oc (un solo comando)
oc expose service my-service --hostname=myapp.apps.cluster.example.com

# Output atteso:
# route.route.openshift.io/my-service exposed

```

3.2 Source-to-Image (S2I)

S2I consente di costruire immagini container direttamente dal codice sorgente senza scrivere un Dockerfile:

```

# Deploy diretto da repository Git (Node.js)
oc new-app nodejs~https://github.com/myorg/my-nodejs-app.git \
  --name=my-app \
  --env=NODE_ENV=production

# Output atteso:
# --> Found image nodejs:18 (builder image)
# --> Creating resources ...
#   imagestream.image.openshift.io "my-app" created
#   buildconfig.build.openshift.io "my-app" created
#   deployment.apps "my-app" created

```

```
# service "v1" created
# --> Success

# Verificare lo stato della build
oc get builds
oc logs -f bc/my-app
```

3.3 SecurityContextConstraints (SCC)

Le SCC di OpenShift sono più granulari rispetto al Pod Security Admission di Kubernetes:

```
# Visualizzare le SCC disponibili nel cluster
oc get scc
# Output atteso:
```

# NAME	PRIV	CAPS	SELINUX	RUNASUSER
FSGROUP				
# anyuid	false	<no value>	MustRunAs	RunAsAny
RunAsAny				
# hostaccess	false	<no value>	MustRunAs	MustRunAs
MustRunAs				
# hostmount-anyuid	false	<no value>	MustRunAs	RunAsAny
RunAsAny				
# hostnetwork	false	<no value>	MustRunAs	MustRunAs
MustRunAs				
# nonroot	false	<no value>	MustRunAs	
MustRunAsNonRoot				RunAsAny
# privileged	true	[*]	RunAsAny	RunAsAny
RunAsAny				
# restricted	false	<no value>	MustRunAs	
MustRunAsRange				MustRunAs

```
# Assegnare una SCC a un ServiceAccount
oc adm policy add-scc-to-user anyuid -z my-service-account -n my-namespace
```

3.4 CLI `oc` vs `kubectl`

`oc` è un superset di `kubectl`: tutti i comandi `kubectl` funzionano con `oc`, ma `oc` aggiunge comandi specifici OCP:

```
# Comandi esclusivi di oc (non disponibili in kubectl)

# Gestione Project (namespace con policy aggiuntive)
oc new-project my-project --description="Il mio progetto" --display-name="My Project"

# Login e gestione contesti
oc login https://api.cluster.example.com:6443

# Build e deploy rapido
```

```
oc new-app python~https://github.com/myorg/myapp.git

# Esposizione di un servizio come Route
oc expose svc/my-service

# Rollout management avanzato
oc rollout status dc/my-deploymentconfig
oc rollout history dc/my-deploymentconfig
oc rollback dc/my-deploymentconfig

# Debug di un nodo
oc debug node/worker-0.cluster.example.com

# Rsh in un Pod (Remote Shell)
oc rsh my-pod-name
```

4. Vantaggi di OCP in Contesto Enterprise

4.1 Sicurezza by Default

OpenShift applica policy di sicurezza restrittive **out-of-the-box**:

- I container **non possono girare come root** per default (SCC **restricted**)
- SELinux è abilitato e configurato automaticamente
- Tutte le comunicazioni tra componenti del cluster usano **mTLS**
- Image scanning integrato tramite Red Hat Quay (opzionale)

4.2 Gestione del Ciclo di Vita

- Aggiornamenti **OTA** (Over-The-Air) con rollback automatico in caso di errore
- Canali di aggiornamento: **stable**, **fast**, **candidate**, **eus** (Extended Update Support)
- Gestione degli Operator tramite **OLM** con versioning e dipendenze

4.3 Supporto Enterprise Red Hat

- SLA garantito con Red Hat (supporto 24/7 disponibile)
- Patch di sicurezza certificate e testate
- Compatibilità con il Red Hat Ecosystem (RHEL, Quay, ACM, ACS)

4.4 Integrazione con Azure ARO

In ARO i vantaggi enterprise si amplificano con l'integrazione nativa Azure:

```
# Integrare ARO con Azure Container Registry (ACR)
az aro update \
  --name myAROCluster \
  --resource-group myResourceGroup \
  --acr myRegistry.azurecr.io
```

```
# Configurare Azure Active Directory come Identity Provider
# (tramite console web: Administration → Cluster Settings → Configuration → OAuth)
```

5. Quando Scegliere OCP invece di Kubernetes Vanilla

Scenario	Kubernetes Vanilla	OpenShift
Startup / prototipazione	✓ Leggero, flessibile	Eccessivo
Enterprise con compliance	Richiede molte configurazioni	✓ Sicuro by default
Team DevOps esperto	✓ Pieno controllo	✓ Anche
Team con skill limitati	Curva ripida	✓ Console + S2I semplificano
Multi-tenant sicuro	Configurazione complessa	✓ SCC + Project isolation
CI/CD integrato	Tools esterni necessari	✓ Pipelines + GitOps integrati
Supporto SLA richiesto	Solo community	✓ Red Hat enterprise support

6. Riepilogo dei Punti Chiave

- OpenShift è una **distribuzione enterprise di Kubernetes**, non un'alternativa: include tutta l'API Kubernetes
- La CLI **oc** è un **superset di kubectl**: ogni comando **kubectl** funziona anche con **oc**
- OCP aggiunge: Route, S2I, SCC, console web, monitoring, logging, OLM, Pipelines, GitOps
- La sicurezza è **restrittiva per default** (container non-root, SELinux, mTLS)
- In contesto enterprise OCP riduce il **time-to-production** grazie a componenti integrati e supporto Red Hat

7. Domande di Verifica

1. Un amministratore Kubernetes conosce **kubectl**. Deve reimparare tutti i comandi per lavorare su OpenShift? Perché?
2. Qual è la differenza principale tra un **Ingress** Kubernetes e una **Route** OpenShift in termini di gestione TLS?
3. Cosa impedisce a un container di girare come root per default in OpenShift? Quale oggetto applica questo controllo?
4. In quale scenario preferiresti Kubernetes vanilla rispetto a OpenShift? Motiva la risposta.
5. Cosa si intende per "aggiornamento OTA" in OCP e qual è il vantaggio rispetto a un aggiornamento manuale?

Riferimenti

- [OpenShift vs Kubernetes - Red Hat Blog](#)
- [OCP Security Context Constraints](#)

- [Source-to-Image \(S2I\)](#)
- [OpenShift Route Documentation](#)